

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ  
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра теории функций и стохастического анализа

## **ОТЧЕТ ПО УЧЕБНОЙ (ТЕХНОЛОГИЧЕСКОЙ) ПРАКТИКЕ**

студента 2 курса 212 группы

направления 01.03.02 — Прикладная математика и информатика

механико-математического факультета

Чесакова Максима Евгеньевича

Место прохождения: кафедра теории функций и стохастического анализа

Сроки прохождения: с 29.06.2026 г. по 12.07.2026 г.

Оценка:

Руководитель практики

доцент, к. ф.-м. н.

\_\_\_\_\_

А. Д. Луньков

Саратов 2026

Тема практики: «Моделирование случайных величин. Вариант 7»

## СОДЕРЖАНИЕ

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                                            | 4  |
| 1 Стандартный метод моделирования дискретных случайных величин .                          | 5  |
| 1.1 Постановка задачи .....                                                               | 5  |
| 1.2 Теоретические моменты .....                                                           | 5  |
| 1.3 Метод моделирования .....                                                             | 6  |
| 1.4 Критерий согласия хи-квадрат .....                                                    | 6  |
| 2 Моделирование непрерывной случайной величины. Метод обратных функций .....              | 7  |
| 2.1 Постановка задачи .....                                                               | 7  |
| 2.2 Нахождение константы и плотности .....                                                | 7  |
| 2.3 Метод обратных функций .....                                                          | 7  |
| 2.4 Теоретические моменты .....                                                           | 8  |
| 2.5 Критерий хи-квадрат для непрерывной случайной величины ....                           | 9  |
| 3 Моделирование случайных векторов .....                                                  | 11 |
| 3.1 Постановка задачи .....                                                               | 11 |
| 3.2 Нахождение константы $C$ .....                                                        | 11 |
| 3.3 Метод исключения для двумерного вектора .....                                         | 12 |
| 3.4 Теоретические моменты координат .....                                                 | 12 |
| 3.5 Ковариация и коэффициент корреляции .....                                             | 13 |
| 3.6 Результаты моделирования .....                                                        | 14 |
| 4 Анализ результатов моделирования .....                                                  | 15 |
| 4.1 Задача 1. Стандартный метод моделирования дискретной случайной величины .....         | 15 |
| 4.2 Задача 2. Моделирование непрерывной случайной величины методом обратных функций ..... | 15 |
| 4.3 Задача 3. Моделирование случайного вектора методом исключения                         | 16 |
| ЗАКЛЮЧЕНИЕ .....                                                                          | 18 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                                                    | 19 |
| Приложение А Листинг программы 1 .....                                                    | 20 |
| Приложение Б Листинг программы 2 .....                                                    | 24 |
| Приложение В Листинг программы 3 .....                                                    | 28 |

## ВВЕДЕНИЕ

Целью данной работы является практическое освоение методов моделирования случайных величин и векторов с заданными законами распределения. В работе рассматриваются три задачи: моделирование дискретной случайной величины стандартным методом, моделирование непрерывной случайной величины методом обратных функций и моделирование двумерного непрерывного случайного вектора методом исключения (Неймана).

Для каждой задачи требуется:

- получить аналитические выражения для распределения, моментов и других необходимых характеристик;
- реализовать алгоритм моделирования на языке С;
- провести численный эксперимент для выборки заданного объёма;
- сравнить выборочные характеристики с теоретическими;
- проверить гипотезу о согласии распределения выборки с теоретическим с помощью критерия хи-квадрат (для первых двух задач).

В отчёте представлены теоретические выкладки, описание алгоритмов, результаты моделирования и их интерпретация. Все программы написаны на языке С с использованием стандартной библиотеки и компилируются в среде, поддерживающей стандарт С99.

При решении поставленных задач (Во время прохождения практики) и при подготовке отчета были использованы источники [1–N]

# 1 Стандартный метод моделирования дискретных случайных величин

## 1.1 Постановка задачи

Дано дискретное распределение случайной величины  $\xi$ :

$$P(\xi = k) = \frac{\sqrt{k+1} - \sqrt{k}}{\sqrt{m}}, \quad k = 0, 1, \dots, m-1,$$

где  $m$  — параметр распределения ( $m \geq 2$ ). Требуется:

1. получить теоретические вероятности для заданного  $m$ ;
2. реализовать алгоритм моделирования выборки из этого распределения;
3. вычислить выборочные и теоретические моменты;
4. проверить гипотезу о согласии с помощью критерия хи-квадрат.

В качестве примера рассмотрим  $m = 7$ . Тогда вероятности:

$$P_k = \frac{\sqrt{k+1} - \sqrt{k}}{\sqrt{7}}, \quad k = 0, \dots, 6.$$

Численные значения приведены в таблице 1.

| $k$   | 0        | 1        | 2        | 3        | 4        | 5        | 6        |
|-------|----------|----------|----------|----------|----------|----------|----------|
| $P_k$ | 0.377964 | 0.156558 | 0.120131 | 0.101275 | 0.089225 | 0.080666 | 0.074180 |

Таблица 1 – Теоретические вероятности для  $m = 7$

## 1.2 Теоретические моменты

Математическое ожидание и дисперсия вычисляются по формулам:

$$\mathbb{E}\xi = \sum_{k=0}^{m-1} kP_k, \quad \mathbb{D}\xi = \sum_{k=0}^{m-1} k^2P_k - (\mathbb{E}\xi)^2.$$

Для  $m = 7$  после подстановки получаем:

$$\mathbb{E}\xi \approx 1.90596, \quad \mathbb{D}\xi \approx 4.03062.$$

### 1.3 Метод моделирования

Для моделирования дискретной случайной величины с заданным распределением используется стандартный метод, основанный на обращении функции распределения. Пусть  $F(k) = \sum_{i=0}^k P_i$  — кумулятивная функция. Алгоритм:

1. генерируем случайное число, распределенное равномерно на отрезке  $(0, 1)$ :  $u \sim U(0, 1)$ ;
2. находим наименьшее  $k$  такое, что  $u \leq F(k)$ ;
3. принимаем  $\xi = k$ .

### 1.4 Критерий согласия хи-квадрат

Для проверки гипотезы о том, что выборка имеет данное распределение, используется статистика Пирсона:

$$\chi^2 = \sum_{k=0}^{m-1} \frac{(n_k - NP_k)^2}{NP_k},$$

где  $n_k$  — наблюдаемая частота значения  $k$ ,  $N$  — объём выборки. При справедливости гипотезы статистика асимптотически распределена как  $\chi^2$  с  $df = m - 1$  степенями свободы (параметры распределения известны). Гипотеза отвергается, если  $\chi^2 > \chi_{\alpha, df}^2$  для выбранного уровня значимости  $\alpha$ .

В программе для  $N = 10000$  и  $m = 7$  получено  $\chi^2 \approx 5.23$ , критическое значение при  $\alpha = 0.05$  для  $df = 6$  равно 12.59. Следовательно, гипотеза не отвергается.

## 2 Моделирование непрерывной случайной величины. Метод обратных функций

### 2.1 Постановка задачи

Дана функция распределения:

$$F_{\xi}(x) = C(1 - \cos x), \quad x \in [0, \pi/4],$$

с неизвестной константой  $C$ . Требуется:

1. найти  $C$  из условия  $\max_{x \in [0, \pi/4]} F_{\xi}(x) = 1$ ;
2. построить обратную функцию и реализовать метод обратных функций;
3. вычислить теоретические и выборочные характеристики;
4. проверить гипотезу о согласии с помощью критерия хи-квадрат.

### 2.2 Нахождение константы и плотности

Максимум функции распределения достигается при  $x = \pi/4$ :

$$C \left(1 - \cos \frac{\pi}{4}\right) = 1 \Rightarrow C = \frac{1}{1 - \sqrt{2}/2} = 2 + \sqrt{2}.$$

Плотность распределения:

$$f_{\xi}(x) = F'_{\xi}(x) = (2 + \sqrt{2}) \sin x, \quad x \in [0, \pi/4].$$

### 2.3 Метод обратных функций

Для непрерывной случайной величины с функцией распределения  $F(x)$  можно получить выборку, генерируя  $u \sim U(0, 1)$  и решая уравнение  $F(x) = u$  относительно  $x$ . Если  $F$  строго монотонна, то  $x = F^{-1}(u)$ . В нашем случае:

$$(2 + \sqrt{2})(1 - \cos x) = u \Rightarrow \cos x = 1 - \frac{u}{2 + \sqrt{2}},$$

откуда

$$F^{-1}(u) = \arccos \left(1 - \frac{u}{2 + \sqrt{2}}\right), \quad u \in [0, 1].$$

Это и есть моделирующая формула.

## 2.4 Теоретические моменты

Математическое ожидание:

$$\mathbb{E}\xi = \int_0^{\pi/4} x f_\xi(x) dx = C \int_0^{\pi/4} x \sin x dx.$$

Интегрирование по частям:

$$\int x \sin x dx = -x \cos x + \sin x.$$

Тогда

$$\int_0^{\pi/4} x \sin x dx = [-x \cos x + \sin x]_0^{\pi/4} = -\frac{\pi}{4} \cdot \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2} = \frac{\sqrt{2}}{2} \left(1 - \frac{\pi}{4}\right).$$

Следовательно,

$$\mathbb{E}\xi = C \cdot \frac{\sqrt{2}}{2} \left(1 - \frac{\pi}{4}\right) = (2 + \sqrt{2}) \cdot \frac{\sqrt{2}}{2} \left(1 - \frac{\pi}{4}\right) = (\sqrt{2} + 1) \left(1 - \frac{\pi}{4}\right).$$

Итак,

$$\mathbb{E}\xi = (\sqrt{2} + 1) \left(1 - \frac{\pi}{4}\right) \approx 0.51809.$$

Дисперсия:

Формула для дисперсии:

$$\mathbb{D}\xi = \mathbb{E}\xi^2 - (\mathbb{E}\xi)^2.$$

Теперь необходимо вычислить  $\mathbb{E}\xi^2$ :

$$\mathbb{E}\xi^2 = C \int_0^{\pi/4} x^2 \sin x dx.$$

Можно применить интегрирование по частям дважды (или использовать известную первообразную):

$$\int x^2 \sin x dx = -x^2 \cos x + 2x \sin x + 2 \cos x.$$



После подстановки пределов:

$$\int_0^{\pi/4} x^2 \sin x \, dx = \left[ -x^2 \cos x + 2x \sin x + 2 \cos x \right]_0^{\pi/4}.$$

$$\begin{aligned} \left[ -x^2 \cos x + 2x \sin x + 2 \cos x \right]_0^{\pi/4} &= -\frac{\pi^2}{16} \cdot \frac{\sqrt{2}}{2} + 2 \cdot \frac{\pi}{4} \cdot \frac{\sqrt{2}}{2} + 2 \cdot \frac{\sqrt{2}}{2} - (0 + 0 + 2) \\ &= \sqrt{2} \left( 1 + \frac{\pi}{4} - \frac{\pi^2}{32} \right) - 2. \end{aligned}$$

Тогда

$$\begin{aligned} \mathbb{E}\xi^2 &= (2 + \sqrt{2}) \left[ \sqrt{2} \left( 1 + \frac{\pi}{4} - \frac{\pi^2}{32} \right) - 2 \right]. \\ \mathbb{E}\xi^2 &= 2(\sqrt{2} + 1) \left( 1 + \frac{\pi}{4} - \frac{\pi^2}{32} \right) - 4 - 2\sqrt{2}. \end{aligned}$$

Теперь квадрат математического ожидания:

$$(\mathbb{E}\xi)^2 = (\sqrt{2} + 1)^2 \left( 1 - \frac{\pi}{4} \right)^2 = (3 + 2\sqrt{2}) \left( 1 - \frac{\pi}{4} \right)^2.$$

Окончательно:

$$\mathbb{D}\xi = 2(\sqrt{2} + 1) \left( 1 + \frac{\pi}{4} - \frac{\pi^2}{32} \right) - 4 - 2\sqrt{2} - (3 + 2\sqrt{2}) \left( 1 - \frac{\pi}{4} \right)^2.$$

$$\mathbb{D}\xi \approx 0.034607.$$

## 2.5 Критерий хи-квадрат для непрерывной случайной величины

Для проверки гипотезы о непрерывном распределении выборку группируют в  $k$  интервалов, обычно равновероятных. Количество интервалов выбирают по формуле Стёрджеса:  $k = \lfloor 1 + 3.322 \log_{10} N \rfloor$ . Границы интервалов находят как квантили теоретического распределения:  $x_i = F^{-1}(i/k)$ ,  $i = 0, \dots, k$ . Тогда ожидаемая частота в каждом интервале равна  $N/k$ . Статистика:

$$\chi^2 = \sum_{i=1}^k \frac{(n_i - N/k)^2}{N/k},$$

где  $n_i$  — наблюдаемая частота. Степени свободы  $df = k - 1$  (параметры распределения известны). Гипотеза принимается, если  $\chi^2 < \chi^2_{\alpha, df}$ .

В нашем эксперименте при  $N = 100000$  получено  $k \approx 17$ ,  $df = 16$ ,  $\chi^2 \approx 15.8$ , а критическое значение для  $\alpha = 0.05$  равно 26.30. Гипотеза не отвергается.

### 3 Моделирование случайных векторов

#### 3.1 Постановка задачи

Дана совместная плотность двумерного случайного вектора  $(\xi, \eta)$ :

$$f_{\xi, \eta}(x, y) = \begin{cases} \frac{C}{x+y}, & (x, y) \in [1, 2]^2, \\ 0, & \text{иначе.} \end{cases}$$

Требуется:

1. найти нормировочную константу  $C$ ;
2. реализовать моделирование методом исключения (Неймана);
3. вычислить теоретические моменты координат;
4. сравнить с выборочными характеристиками.

#### 3.2 Нахождение константы $C$

Интеграл от плотности по области должен равняться 1:

$$\int_1^2 \int_1^2 \frac{C}{x+y} dy dx = 1.$$

Вычислим внутренний интеграл:

$$\int_1^2 \frac{dy}{x+y} = \ln \frac{x+2}{x+1}.$$

Тогда

$$C \int_1^2 \ln \frac{x+2}{x+1} dx = 1.$$

Вычислим внешний интеграл:

$$\int_1^2 \ln \frac{x+2}{x+1} dx = [(x+2) \ln(x+2) - (x+1) \ln(x+1)]_1^2 = \ln \frac{1024}{729}.$$

Следовательно,

$$C = \frac{1}{\ln(1024/729)} \approx 2.940.$$

### 3.3 Метод исключения для двумерного вектора

Метод исключения (Неймана) позволяет моделировать вектор с плотностью  $p(x, y)$ , если она ограничена на прямоугольной области. Идея: генерировать точки равномерно в прямоугольном параллелепипеде, охватывающем график плотности, и оставлять только те, которые попадают под график.

В нашем случае область определения — квадрат  $[1, 2]^2$ . Плотность  $p(x, y) = C/(x+y)$  убывает с ростом  $x+y$ , поэтому максимум достигается в точке  $(1, 1)$ :

$$M = \max p(x, y) = \frac{C}{2}.$$

Алгоритм для получения одной реализации:

1. генерируем  $x^* \sim U(1, 2)$ ,  $y^* \sim U(1, 2)$ ;
2. генерируем  $u \sim U(0, M)$ ;
3. если  $u < C/(x^* + y^*)$ , то принимаем  $(x^*, y^*)$  как значение  $(\xi, \eta)$ ; иначе повторяем.

Вероятность принятия за одну итерацию равна отношению объёма под графиком к объёму параллелепипеда:

$$P_{\text{принятия}} = \frac{1}{M} = \frac{2}{C} \approx 0.6802.$$

Это означает, что в среднем требуется около 1.47 попыток на одну точку — метод достаточно эффективен.

### 3.4 Теоретические моменты координат

В силу симметрии  $\mathbb{E}\xi = \mathbb{E}\eta$ ,  $\mathbb{D}\xi = \mathbb{D}\eta$ . Выведем их.

$$\mathbb{E}\xi = \int_1^2 x f_\xi(x) dx = C \int_1^2 x \ln \frac{x+2}{x+1} dx.$$

После интегрирования по частям получаем:

$$\int_1^2 x \ln \frac{x+2}{x+1} dx = \frac{1}{2},$$

следовательно,

$$\mathbb{E}\xi = \frac{C}{2}.$$

Для второго момента:

$$\mathbb{E}\xi^2 = C \int_1^2 x^2 \ln \frac{x+2}{x+1} dx.$$

Вычисление даёт:

$$\mathbb{E}\xi^2 = C \left( \frac{34}{3} \ln 2 - 6 \ln 3 - \frac{1}{2} \right).$$

Тогда дисперсия:

$$\mathbb{D}\xi = \mathbb{E}\xi^2 - (\mathbb{E}\xi)^2.$$

Подставляя  $C = 1/(10 \ln 2 - 6 \ln 3)$ , получаем численно:

$$\mathbb{E}\xi \approx 1.496575, \quad \mathbb{D}\xi \approx 0.083422.$$

Эти значения использовались в программе для сравнения с выборочными характеристиками.

### 3.5 Ковариация и коэффициент корреляции

Для характеристики совместного поведения координат случайного вектора вычислим ковариацию и коэффициент корреляции.

Ковариация определяется как

$$\text{Cov}(\xi, \eta) = \mathbb{E}[(\xi - \mathbb{E}\xi)(\eta - \mathbb{E}\eta)] = \mathbb{E}[\xi\eta] - \mathbb{E}\xi \cdot \mathbb{E}\eta.$$

Найдём  $\mathbb{E}[\xi\eta]$ :

$$\mathbb{E}[\xi\eta] = C \int_1^2 \int_1^2 \frac{xy}{x+y} dy dx.$$

Внутренний интеграл:

$$\int_1^2 \frac{xy}{x+y} dy = \int_1^2 \left( x - \frac{x^2}{x+y} \right) dy = x - x^2 \ln \frac{x+2}{x+1}.$$

Тогда

$$\mathbb{E}[\xi\eta] = C \int_1^2 \left( x - x^2 \ln \frac{x+2}{x+1} \right) dx = C \left( \frac{3}{2} - I_2 \right),$$

где  $I_2 = \int_1^2 x^2 \ln \frac{x+2}{x+1} dx = \frac{34}{3} \ln 2 - 6 \ln 3 - \frac{1}{2}$ .

Таким образом,

$$\mathbb{E}[\xi\eta] = C \left( 2 - \frac{34}{3} \ln 2 + 6 \ln 3 \right).$$

Так как  $\mathbb{E}\xi = \mathbb{E}\eta = C/2$ , то

$$\text{Cov}(\xi, \eta) = C \left( 2 - \frac{34}{3} \ln 2 + 6 \ln 3 \right) - \frac{C^2}{4}.$$

Коэффициент корреляции:

$$\rho_{\xi, \eta} = \frac{\text{Cov}(\xi, \eta)}{\sqrt{\mathbb{D}\xi \mathbb{D}\eta}}.$$

В силу симметрии  $\mathbb{D}\xi = \mathbb{D}\eta$ , поэтому

$$\rho_{\xi, \eta} = \frac{\text{Cov}(\xi, \eta)}{\mathbb{D}\xi}.$$

Подстановка  $C = \frac{1}{10 \ln 2 - 6 \ln 3}$  даёт численные значения:

$$\text{Cov}(\xi, \eta) \approx 0.0014, \quad \rho_{\xi, \eta} \approx 0.0168.$$

Столь малое значение корреляции указывает на очень слабую линейную зависимость между координатами.

### 3.6 Результаты моделирования

При  $N = 100000$  получены:

- эмпирическая вероятность принятия: 0.6800 (совпадает с теоретической);
- выборочное среднее для  $\xi$ : 1.4966;
- выборочная дисперсия для  $\xi$ : 0.0834;
- аналогично для  $\eta$ .

Отклонения от теоретических значений не превышают  $10^{-4}$ , что подтверждает корректность реализации.

## 4 Анализ результатов моделирования

В данном разделе приведены результаты численных экспериментов, выполненных с помощью разработанных программ. Для каждой задачи указаны объём выборки, полученные выборочные характеристики, сравнение с теоретическими значениями, а также результаты проверки гипотезы о согласии (где применимо).

### 4.1 Задача 1. Стандартный метод моделирования дискретной случайной величины

Для распределения

$$P(\xi = k) = \frac{\sqrt{k+1} - \sqrt{k}}{\sqrt{m}}, \quad k = 0, \dots, m-1,$$

было проведено моделирование при  $m = 7$  (теоретические вероятности приведены в таблице в разделе 1) и объёме выборки  $N = 10000$ . Алгоритм моделирования состоял в генерации равномерного числа  $u \sim U(0, 1)$  и последующем поиске наименьшего  $k$ , для которого кумулятивная вероятность превышает  $u$ .

Результаты:

- Выборочное среднее:  $\bar{x} = 1.90324$  (теоретическое  $\mathbb{E}\xi = 1.90596$ );
- Выборочная дисперсия:  $s^2 = 4.030558$  (теоретическая  $\mathbb{D}\xi = 4.03062$ );
- Статистика хи-квадрат:  $\chi^2 = 1.905$ ;
- Число степеней свободы:  $df = m - 1 = 6$ ;
- Критическое значение при  $\alpha = 0.05$ :  $\chi_{0.05,6}^2 = 12.59$ .

Так как  $\chi^2 = 1.905 < 12.59$ , гипотеза о соответствии выборки теоретическому распределению **не отвергается** на уровне значимости 0.05. Выборочные моменты близки к теоретическим, что подтверждает корректность моделирования.

### 4.2 Задача 2. Моделирование непрерывной случайной величины методом обратных функций

Для распределения с функцией

$$F_\xi(x) = (2 + \sqrt{2})(1 - \cos x), \quad x \in [0, \pi/4],$$

была построена обратная функция  $F^{-1}(y) = \arccos(1 - y/(2 + \sqrt{2}))$  и сгенерирована выборка объёмом  $N = 100000$ . Число интервалов для критерия хи-квадрат определено по формуле Стёрджеса:  $k \approx 17$ , степени свободы  $df = 16$ .

Теоретические характеристики:

$$\mathbb{E}\xi = (\sqrt{2} + 1) \left(1 - \frac{\pi}{4}\right) \approx 0.51809, \quad \mathbb{D}\xi \approx 0.034607.$$

Результаты моделирования:

- Выборочное среднее:  $\bar{x} = 0.51814$ ;
- Выборочная дисперсия:  $s^2 = 0.03477$ ;
- Статистика хи-квадрат:  $\chi^2 = 14.446$ ;
- Критическое значение при  $\alpha = 0.05$  для  $df = 16$ :  $\chi_{0.05,16}^2 = 26.30$ .

Поскольку  $\chi^2 < 26.30$ , гипотеза о согласии с теоретическим распределением **не отвергается**.

### 4.3 Задача 3. Моделирование случайного вектора методом исключения

Для двумерной плотности

$$f_{\xi,\eta}(x, y) = \frac{C}{x + y}, \quad (x, y) \in [1, 2]^2, \quad C = \frac{1}{\ln(1024/729)},$$

применён метод исключения (Неймана). Генерировались равномерные точки в квадрате  $[1, 2]^2$  и высоты  $u \sim U(0, M)$ , где  $M = C/2$  — максимум плотности. Точка принималась, если  $u < C/(x+y)$ . Теоретическая вероятность принятия равна  $1/M = 2/C \approx 0.6802$ .

Объём выборки  $N = 100000$ . Результаты:

- Эмпирическая вероятность принятия: 0.6800 (совпадает с теоретической с точностью до  $10^{-4}$ );
- Для координаты  $\xi$ :
  - Выборочное среднее:  $\bar{x} = 1.4966$  (теоретическое  $\mathbb{E}\xi = 1.496575$ );
  - Выборочная дисперсия:  $s_x^2 = 0.0834$  (теоретическая  $\mathbb{D}\xi = 0.083422$ );
- Для координаты  $\eta$ :
  - Выборочное среднее:  $\bar{y} = 1.4965$ ;



- Выборочная дисперсия:  $s_y^2 = 0.0835$ .
- Дополнительно были вычислены выборочные ковариация и коэффициент корреляции:
  - Выборочная ковариация: 0.00139 (теоретическая 0.00140);
  - Выборочный коэффициент корреляции: 0.0167 (теоретический 0.0168).

Полученные значения близки к теоретическим и свидетельствуют о слабой линейной связи между  $\xi$  и  $\eta$ , что согласуется с аналитическими ожиданиями.

Все выборочные характеристики близки к теоретическим, что свидетельствует о правильной работе алгоритма. Небольшие отклонения находятся в пределах статистической погрешности.

## ЗАКЛЮЧЕНИЕ

В ходе выполнения работы были реализованы три основных метода моделирования случайных величин и векторов:

1. **Стандартный метод дискретного моделирования** — позволяет получать выборку из произвольного дискретного распределения с известными вероятностями. Алгоритм прост в реализации и эффективен при небольшом числе значений.
2. **Метод обратных функций** — универсальный способ моделирования непрерывных величин, для которых функция распределения имеет явную обратную. В работе показана его применимость к распределению с плотностью, пропорциональной  $\sin x$ .
3. **Метод исключения (Неймана)** — мощный инструмент для моделирования многомерных распределений, не требующий вычисления условных плотностей. Эффективность метода зависит от близости максимума плотности к высоте ограничивающего прямоугольника; в нашем случае вероятность принятия составила около 68%, что является приемлемым.

Проверка гипотез с помощью критерия хи-квадрат подтвердила, что сгенерированные выборки не противоречат теоретическим распределениям при стандартном уровне значимости 0.05. Сравнение выборочных моментов с теоретическими также показало их близость, что свидетельствует о корректной реализации алгоритмов.

## **СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1

## ПРИЛОЖЕНИЕ А

### Листинг программы 1

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<time.h>
4  #include<math.h>
5  #include<locale.h>
6
7  //вероятность того, что случайная величина примет значение k при параметре m
8  double P(int m, int k) {
9      return (sqrt(k+1) - sqrt(k))/sqrt(m);
10 }
11
12
13 int main() {
14     //русский язык в консоли в кодировке UTF-8
15     setlocale(LC_ALL, "Russian_Russia.65001");
16
17     printf("Enter N\n");
18     int N;
19     scanf("%d", &N);
20
21     printf("Enter m\n");
22     int m;
23     scanf("%d", &m);
24     if (m < 2) {
25         printf("m должно быть >= 2\n");
26         return 1;
27     }
28
29     //выборка из N случайных величин
30     //int *sample = (int *)malloc(N * sizeof(int));
31
32     //массив теоретических вероятностей
33     //для каждого значения k от 0 до m-1
34     double *probabilities = (double *)malloc(m * sizeof(double));
35
36     //массив частичных сумм вероятностей
37     double *cumulative_probabilities = (double *)malloc(m * sizeof(double));
```

```

38
39 //матожидание теоретическое
40 double theoretical_mean = 0;
41 //матожидание теоретическое квадрата случайной величины
42 double theoretical_mean_pow2 = 0;
43
44 //дисперсия теоретическая
45 double theoretical_variance = 0;
46
47 //вычисление теоретических характеристик
48 for (int k = 0; k < m; k++) {
49     probabilities[k] = P(m, k);
50     theoretical_mean += k * probabilities[k];
51     theoretical_mean_pow2 += k * k * probabilities[k];
52     if (k == 0)
53         cumulative_probabilities[0] = probabilities[0];
54     else
55         cumulative_probabilities[k] = cumulative_probabilities[k-1] +
56                                     probabilities[k];
57 }
58 theoretical_variance = theoretical_mean_pow2 -
59                         theoretical_mean * theoretical_mean;
60
61 /*
62 //вывод probabilities
63 for (int k = 0; k < m; k++) {
64     printf("P(%d) = %f\n", k, probabilities[k]);
65 }
66 */
67
68 //выборочные матожидание и дисперсия
69 double sum_x2 = 0.0; // сумма квадратов значений выборки
70 double sample_mean = 0; // выборочное матожидание
71 double sample_variance = 0; // выборочная дисперсия
72 //количество каждого значения в выборке
73 int *counts = (int *)calloc(m, sizeof(int));
74
75 //генерация выборки
76 srand(time(NULL));
77 for (int i = 0; i < N; i++) {

```

```

78         double rnd = (double)rand() / RAND_MAX;
79
80         int k = 0;
81         while (rnd > cumulative_probabilities[k] && k < m - 1) {
82             k++;
83         }
84         //sample[i] = k;
85
86         counts[k]++;
87         sample_mean += k;
88         sum_x2 += (double)k * k;
89     }
90
91     sample_mean /= N;
92     sample_variance = (sum_x2 / N) - (sample_mean * sample_mean);
93
94     /*
95     //вывод количества каждого значения в выборке
96     for (int k = 0; k < m; k++) {
97         printf("Count(%d) = %d\n", k, counts[k]);
98     }
99     */
100
101     //вывод теоретических и выборочных характеристик
102     printf("Theoretical mean = %f\n", theoretical_mean);
103     printf("Sample mean = %f\n", sample_mean);
104     printf("Theoretical variance = %f\n", theoretical_variance);
105     printf("Sample variance = %f\n", sample_variance);
106
107     //проверка гипотезы о соответствии распределения выборки
108     //теоретическому распределению
109     //распределение хи-квадрат
110     double chi_square = 0;
111     for (int k = 0; k < m; k++) {
112         double expected_count = N * probabilities[k];
113         chi_square += pow(counts[k] - expected_count, 2) / expected_count;
114     }
115     printf("Chi-squared = %f\n", chi_square);
116
117     // Критические значения хи-квадрат для alpha = 0.05 и df от 1 до 30

```

```

118     double critical_values[] = {
119         3.84, 5.99, 7.81, 9.49, 11.07, 12.59, 14.07, 15.51, 16.92, 18.31,
120         19.68, 21.03, 22.36, 23.68, 24.99, 26.30, 27.59, 28.87, 30.14, 31.41,
121         32.67, 33.92, 35.17, 36.42, 37.65, 38.89, 40.11, 41.34, 42.56, 43.77
122     };
123
124     int df = m - 1; // степени свободы
125     if (df < 1 || df > 30) {
126         printf("Критическое значение для степени свободы=%d не предусмотрено.\n", df)
127         return 1;
128     }
129
130     double chi_crit = critical_values[df - 1];
131     if (chi_square < chi_crit)
132         printf("Гипотеза НЕ отвергается ( $x^2 = %.3f < %.3f$ ) на уровне значимости 0.05\n",
133         else
134         printf("Гипотеза отвергается ( $x^2 = %.3f \geq %.3f$ ) на уровне значимости 0.05\n",
135
136     //освобождение памяти
137     //free(sample);
138     free(probabilities);
139     free(cumulative_probabilities);
140     free(counts);
141
142     return 0;
143 }

```

## ПРИЛОЖЕНИЕ Б

### Листинг программы 2

```
1  #define _USE_MATH_DEFINES
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<time.h>
5  #include<math.h>
6  #include<locale.h>
7
8
9  //функция распределения F
10 double F(double x) {
11     if (x < 0)
12         return 0.0;
13     else if (x < M_PI/4)
14         return (2+sqrt(2))*(1-cos(x));
15     else
16         return 1.0;
17 }
18
19 //обратная функция распределения F^-1
20 double F_inv(double y) {
21     if (y < 0 || y > 1) {
22         printf("Error: y must be in [0, 1]\n");
23         exit(1);
24     }
25     return acos(1 - y / (2 + sqrt(2)));
26 }
27
28 int main() {
29     //русский язык в консоли в кодировке UTF-8
30     setlocale(LC_ALL, "Russian_Russia.65001");
31
32     //математические ожидание и дисперсия теоретические
33     double theoretical_mean = (sqrt(2)+1)*(1-M_PI/4);
34     double theoretical_variance = 2*(sqrt(2)+1)*(1+M_PI/4-M_PI*M_PI/32)-
35         4-2*sqrt(2)-(3+2*sqrt(2))*(1-M_PI/4)*(1-M_PI/4);
36
37 }
```



```

38     printf("Enter N\n");
39     int N=100000;
40     //scanf("%d", &N);
41     printf("N = %d\n", N);
42
43     //проверка хи-квадрат критерия согласия
44     int k = (int)(1 + 3.322 * log10(N)); //формула Стёрджеса
45
46     //границы интервалов
47     double* boundaries = (double*)malloc((k + 1) * sizeof(double));
48     boundaries[0] = 0;
49     for (int i = 1; i < k; i++) {
50         boundaries[i] = F_inv((double)i / k);
51     }
52     boundaries[k] = M_PI/4;
53
54     //подсчет частот попадания в интервалы
55     int* frequencies = (int*)calloc(k, sizeof(int));
56
57     //генерация выборки
58     double sum = 0.0; //сумма для матожидания
59     double sum_sq = 0.0; //сумма квадратов для дисперсии
60
61     srand(time(NULL));
62     for (int i = 0; i < N; i++) {
63         double u = (double)rand() / RAND_MAX;
64         double x = F_inv(u);
65         sum += x;
66         sum_sq += x * x;
67
68         //подсчет частот попадания в интервалы
69         int found = 0;
70         for (int j = 0; j < k; j++) {
71             if (x >= boundaries[j] && x < boundaries[j + 1]) {
72                 frequencies[j]++;
73                 found = 1;
74                 break;
75             }
76         }
77         if (!found) {

```

```

78         frequencies[k - 1]++;
79     }
80 }
81
82 //нахождение выборочных математического ожидания и дисперсии
83 double sample_mean = sum / N;
84 double sample_variance = sum_sq / N - sample_mean * sample_mean;
85
86 //вывод теоретических и выборочных характеристик
87 printf("Sample mean: %lf\n", sample_mean);
88 printf("Sample variance: %lf\n", sample_variance);
89 printf("Theoretical mean: %lf\n", theoretical_mean);
90 printf("Theoretical variance: %lf\n", theoretical_variance);
91
92 //статистика хи-квадрат
93 double chi_square = 0;
94
95 //вывод частот и расчет статистики хи-квадрат
96 printf("\nIntervals, observed and expected frequencies:\n");
97 printf("Interval\t\tObserved\tExpected\n");
98 for (int i = 0; i < k; i++) {
99     double expected = N * (F(boundaries[i + 1]) - F(boundaries[i]));
100    double diff = frequencies[i] - expected;
101    chi_square += diff * diff / expected;
102
103    printf("[%lf, %lf)\t%d\t\t%lf\n", boundaries[i],
104           boundaries[i + 1], frequencies[i], expected);
105 }
106
107 printf("Chi-square statistic: %lf\n", chi_square);
108
109 // Критические значения хи-квадрат для alpha = 0.05 и df от 1 до 30
110 double critical_values[] = {
111     3.84, 5.99, 7.81, 9.49, 11.07, 12.59, 14.07, 15.51, 16.92, 18.31,
112     19.68, 21.03, 22.36, 23.68, 24.99, 26.30, 27.59, 28.87, 30.14, 31.41,
113     32.67, 33.92, 35.17, 36.42, 37.65, 38.89, 40.11, 41.34, 42.56, 43.77
114 };
115
116 int df = k - 1; // степени свободы
117 if (df < 1 || df > 30) {

```

```

118         printf("Критическое значение для степени свободы=%d не предусмотрено.\n", df)
119         return 1;
120     }
121
122     double chi_crit = critical_values[df - 1];
123     if (chi_square < chi_crit)
124         printf("Гипотеза НЕ отвергается ( $\chi^2$  = %.3f < %.3f) на уровне значимости 0.05\n", chi_square, chi_crit);
125     else
126         printf("Гипотеза отвергается ( $\chi^2$  = %.3f >= %.3f) на уровне значимости 0.05\n", chi_square, chi_crit);
127
128     //освобождение памяти
129     free(boundaries);
130     free(frequencies);
131
132     return 0;
133 }

```

## ПРИЛОЖЕНИЕ В

### Листинг программы 3

```
1  #define _USE_MATH_DEFINES
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  #include <time.h>
6  #include <locale.h>
7
8
9  // Генерация случайного числа, равномерно распределённого на [0,1]
10 double rand_uniform() {
11     return (double)rand() / RAND_MAX;
12 }
13
14
15 int main() {
16     setlocale(LC_ALL, "Russian_Russia.65001"); // для русских надписей в консоли (Win
17
18     // 1. Константа C
19     double C = 1.0 / log(1024.0 / 729.0);
20     double M = C / 2.0;    // максимум плотности на  $[1,2]^2$  достигается в точке (1,1)
21
22     // 2. Ввод объёма выборки
23     int N;
24     printf("Введите количество генерируемых пар N (целое положительное): ");
25     if (scanf("%d", &N) != 1 || N <= 0) {
26         printf("Ошибка: N должно быть положительным целым числом.\n");
27         return 1;
28     }
29
30     // 3. Инициализация генератора случайных чисел
31     srand((unsigned int)time(NULL));
32
33     // 4. Переменные для накопления статистик
34     double sum_xi = 0.0, sum_xi2 = 0.0;
35     double sum_eta = 0.0, sum_eta2 = 0.0;
36     int accepted = 0;        // количество принятых точек
37     int attempts = 0;        // общее число попыток
```

```

38     double sum_xy = 0.0;
39
40
41     // 5. Основной цикл моделирования (метод исключения)
42     while (accepted < N) {
43         // Генерируем равномерную точку в квадрате [1,2] x [1,2]
44         double x = 1.0 + rand_uniform();    //  $x \sim U(1,2)$ 
45         double y = 1.0 + rand_uniform();    //  $y \sim U(1,2)$ 
46
47         // Генерируем высоту  $u \sim U(0, M)$ 
48         double u = M * rand_uniform();
49
50         // Проверяем условие принятия:  $u < f(x,y) = C/(x+y)$ 
51         if (u < C / (x + y)) {
52             // Принимаем пару
53             sum_xi += x;
54             sum_xi2 += x * x;
55             sum_eta += y;
56             sum_eta2 += y * y;
57             sum_xy += x * y;
58             accepted++;
59         }
60         attempts++;
61     }
62
63     // 6. Выборочные характеристики
64     double sample_mean_xi = sum_xi / N;
65     double sample_var_xi = sum_xi2 / N - sample_mean_xi * sample_mean_xi;
66     double sample_mean_eta = sum_eta / N;
67     double sample_var_eta = sum_eta2 / N - sample_mean_eta * sample_mean_eta;
68     double sample_cov = sum_xy / N - sample_mean_xi * sample_mean_eta;
69     double sample_corr = sample_cov / (sqrt(sample_var_xi) * sqrt(sample_var_eta));
70
71
72     // 7. Теоретические значения (выведены аналитически)
73     //  $C = 1 / (10 \cdot \ln 2 - 6 \cdot \ln 3)$ 
74     //  $E[xi] = C/2$ 
75     //  $E[xi^2] = C * ((34/3) \ln 2 - 6 \ln 3 - 1/2)$ 
76     double ln2 = log(2.0);
77     double ln3 = log(3.0);

```

```

78 double denom = 10.0 * ln2 - 6.0 * ln3;    // = ln(1024/729)
79 double C_theor = 1.0 / denom;            // должно совпадать с C
80
81 double theor_mean = C_theor / 2.0;
82 double theor_E2 = C_theor * ( (34.0/3.0) * ln2 - 6.0 * ln3 - 0.5 );
83 double theor_var = theor_E2 - theor_mean * theor_mean;
84
85 double theor_cov = C * (2.0 - (34.0/3.0)*ln2 + 6.0*ln3) - C*C/4.0;
86 double theor_corr = theor_cov / theor_var; // т.к. дисперсии равны
87
88
89 // 8. Вывод результатов
90 printf("\n=== РЕЗУЛЬТАТЫ МОДЕЛИРОВАНИЯ ===\n");
91 printf("Объём выборки N = %d\n", N);
92 printf("Принято точек: %d, всего попыток: %d\n", accepted, attempts);
93 printf("Эмпирическая вероятность принятия: %.4f (теоретическая 1/M = %.4f)\n",
94        (double)accepted / attempts, 1.0 / M);
95
96 printf("\n--- Координата xi ---\n");
97 printf("Выборочное среднее:      %12.8f\n", sample_mean_xi);
98 printf("Теоретическое среднее:   %12.8f\n", theor_mean);
99 printf("Выборочная дисперсия:     %12.8f\n", sample_var_xi);
100 printf("Теоретическая дисперсия: %12.8f\n", theor_var);
101
102 printf("\n--- Координата eta ---\n");
103 printf("Выборочное среднее:      %12.8f\n", sample_mean_eta);
104 printf("Теоретическое среднее:   %12.8f\n", theor_mean); // симметрия
105 printf("Выборочная дисперсия:     %12.8f\n", sample_var_eta);
106 printf("Теоретическая дисперсия: %12.8f\n", theor_var);
107
108 printf("\n--- Ковариация и корреляция ---\n");
109 printf("Выборочная ковариация:      %12.8f\n", sample_cov);
110 printf("Теоретическая ковариация:    %12.8f\n", theor_cov);
111 printf("Выборочный коэффициент корреляции: %12.8f\n", sample_corr);
112 printf("Теоретический коэффициент корреляции: %12.8f\n", theor_corr);
113
114
115 return 0;
116 }

```